

Appendix D. Reproducible Report

PPMA Enrollment Trends

Ronnie Chen, Imaan Jahangir, Haedodam Kim

Table of contents

Section 0: Data Cleaning	3
0.1 Setup	3
0.2 Variable Name Standardization	3
0.3 Term Code Parsing	3
0.4 Term Range Filter	4
0.5 Student Enrollment Span	4
0.6 Variable Removal	4
0.7 Flag Variables	5
0.8 Exclusion Summary	8
0.9 Final Datasets	9
Section 1: Are Students Following the Recommended Course Sequence?	10
1.1 Exclude Summer Terms	10
1.2 Extract Student Entry Term	10
1.3 Define Course Offering Season	10
1.4 Filter to Foundational Courses	10
1.5 Calculate Opportunity Number	11
1.6 Define Adherence	11
1.7 Adherence Rate by Course	12
1.8 Semester Distribution by Course	12
1.9 Adherence Rate by Curriculum Era	12
1.10 Prerequisite Analysis: PA 504 and PA 506 Before PA 403	13
Section 2: To What Extent Are Students Taking Multiple Courses Within the Same Semester, and Are There Signs of Course Overload?	14
2.1 Assign Credit Hours	14
2.2 Remove 1-Credit Courses	14
2.3 Calculate Credit Load per Student per Term	14
2.4 Calculate Course Count per Student per Term	15
2.5 Remove Summer Terms	15

2.6 Create Credit Load Bands	15
2.7 Summarize Credit Load Bands by Term	16
2.8 Create Course Count Bands	16
2.9 Summarize Course Count Bands by Term	17
Section 3: How Has Average Time to Degree Completion Changed Over Time? . . .	17
3.1 Identify PPMA Program Completers	17
3.2 Calculate Number of Terms to Degree	18
3.3 Assign Graduation Academic Year	18
3.4 Summarize Time to Degree by Graduation Cohort	19
3.5 Classify Students by Completion Category	19
3.6 Distribution by Graduation Academic Year	20
Section 4: How Have Enrollment Patterns Across Concentrations and Elective Courses Changed Over Time, and What Do These Trends Suggest About Shifts in Student Preferences?	20
4.1 Prepare Analysis Dataset	20
4.2 Standardize Renumbered Courses	21
4.3 Create Academic Year Labels	22
4.4 Add Course Titles	23
4.5 Separate Core and Elective Courses	25
4.6 Assign Topic Areas	25
4.7 Calculate Percent of Students Enrolled in Each Topic by Academic Year . .	28
4.8 Count Courses Offered by Topic and Academic Year	28
Section 5: To What Extent Do Students Deviate From the Recommended Course Plan Across the Three Programs (MPA, MPP, and MSCA)?	29
5.1 Prepare Analysis Dataset	29
5.2 Standardize Renumbered Courses	29
5.3 Identify Valid PPMA Students	30
5.4 Assign Students to Programs	31
5.5 Define Required and Concentration Courses	32
5.6 Flag On-Plan and Off-Plan Courses	32
5.7 Summarize Off-Plan Courses per Student	33
5.8 Summary Table by Program	33
Section 6: How Has Student Participation in Nonprofit Coursework Changed Over Time, and What Explains These Trends?	34
6.1 Create Academic Year Variable	34
6.2 Share of Nonprofit Enrollments by Academic Year	34
6.3 Number of Nonprofit Course Offerings by Academic Year	34
6.4 Total and Nonprofit Enrollment Counts by Academic Year	35
6.5 Summary Table	35
Section 7: To What Extent Do Students Retake Courses, and Which Courses Are Most Frequently Repeated?	36
7.1 Create Sequential Term Order	36
7.2 Identify Course Retakes	36

7.3 Summarize Retakes at the Student Level	37
7.4 Merge Retake Information with Student Dataset	37
7.5 Retake Share Summary	37
7.6 Most Frequently Retaken Courses	38

Section 0: Data Cleaning

0.1 Setup

Load required packages and import the raw dataset.

```
# Required Packages
library(tidyverse)
library(scales)

# Load Dataset
data <- read.csv("Data/PA course rosters - no student IDs or grades 20260125.csv")
```

0.2 Variable Name Standardization

Convert all variable names to lowercase and replace . with _ for consistency.

```
# Standardize variable names: convert to lowercase and replace "." with "_"
names(data) <- names(data) %>%
  tolower() %>%
  gsub("\\\\.", "_", .)
```

0.3 Term Code Parsing

Extract term_year and term_season from the raw term_code variable.

- Season codes: 1 = Spring, 5 = Summer, 8 = Fall

```
# Extract term_year and term_season from term_code
data <- data %>%
  mutate(
    term_year = as.numeric(str_sub(term_code, 2, 5)),
    term_season = case_when(
      str_sub(term_code, 6, 6) == "1" ~ 1, # Spring
      str_sub(term_code, 6, 6) == "5" ~ 2, # Summer
```

```

    str_sub(term_code, 6, 6) == "8" ~ 3, # Fall
    TRUE ~ NA_real_
  )
)

```

0.4 Term Range Filter

Restrict the dataset to terms between **Fall 2015** and **Spring 2025** (inclusive).

```

# Restrict the dataset to terms between Fall 2015 and Spring 2025
data <- data %>%
  filter(
    (term_year > 2015 & term_year < 2025) |
    (term_year == 2015 & term_season == 3) |
    (term_year == 2025 & term_season == 1)
  )

```

0.5 Student Enrollment Span

For each student, identify their first (**start_term**) and last (**graduation_term**) observed enrollment term.

```

# Identify each student's first and last observed enrollment term
data <- data %>%
  group_by(student_code) %>%
  mutate(
    start_term      = min(term_year * 10 + term_season),
    graduation_term = max(term_year * 10 + term_season)
  ) %>%
  ungroup()

```

0.6 Variable Removal

Remove the **schedule_type_code** variable, which is not needed for analysis.

```

# Remove the 'schedule_type_code' variable
data <- data %>%
  select(-schedule_type_code)

```

0.7 Flag Variables

0.7.1 Online Section Flag

Flag individual course sections that are delivered online, based on a predefined list of CRNs.

```
# Define the list of CRNs corresponding to online sections
online_crns <- unique(c(
  23787, 19022, 47321, 47960, 47962, 47445, 47964, 24777, 24778,
  49197, 49205, 49774, 49207, 49775, 49776, 47445, 48763, 48764,
  48765, 49304, 47960, 47962, 47321, 47445, 39643, 47319
))

# Flag: binary indicator for whether a section is an online section
data <- data %>%
  mutate(
    online_section_flag = as.integer(crn %in% online_crns)
  )
```

0.7.2 Online-Only Student Flag

Flag students who enrolled **exclusively** in online sections. These students will be excluded from the final analysis.

```
# Flag: students enrolled ONLY in online sections (exclusion reason 1)
data <- data %>%
  group_by(student_code) %>%
  mutate(
    online_only_flag = as.integer(all(crn %in% online_crns))
  ) %>%
  ungroup()
```

0.7.3 Nonprofit Track Flag

Flag courses associated with the nonprofit track.

```
# Define course numbers associated with the nonprofit track
nonprofit_courses <- c(531, 551, 571, 538, 572, 530, 573, 536)

# Flag: binary indicator for nonprofit-related courses
data <- data %>%
  mutate(
```

```

    nonprofit_flag = as.integer(
      course_subject_code == "PA" & course_number %in% nonprofit_courses
    )
  )
)

```

0.7.4 Discontinued Course Flag

Flag PA courses that are no longer offered.

```

# Define course numbers that are no longer offered
discontinued_courses <- c(520, 526, 589, 592, 595)

# Flag: binary indicator for discontinued PA courses
data <- data %>%
  mutate(
    discontinued_flag = as.integer(
      course_subject_code == "PA" & course_number %in% discontinued_courses
    )
  )
)

```

0.7.5 Renumbered Course Flag

Flag courses that have been renumbered (includes both old and new course numbers).

```

# Define courses that have been renumbered (both old and new numbers)
renumbered_courses <- c(
  401, 400, 402, 407, 403, 410, 421, 415, 422, 462,
  431, 463, 432, 461, 433, 460, 467, 564, 525, 502,
  563, 537, 571, 538, 572, 530, 573, 536
)

# Flag: binary indicator for renumbered courses
data <- data %>%
  mutate(
    renumbered_flag = as.integer(course_number %in% renumbered_courses)
  )
)

```

0.7.6 Curriculum Era Flag

Distinguish students by the curriculum era when they first enrolled.

- 0 = Pre Spring 2025 curriculum

- 1 = Post Spring 2025 curriculum (started Spring 2025 or later)

```
# Flag: curriculum era
# 0 = Pre Spring 2025 curriculum, 1 = Post Spring 2025 curriculum
data <- data %>%
  mutate(
    curriculum_era_flag = as.integer(start_term >= 20251)
  )
```

0.7.7 Core Course Flag

Flag whether a course is a core requirement or an elective.

```
# Define core courses
core_courses <- c(
  401, 402, 403, 431, 433, 434, 446, 470,
  504, 505, 506, 515, 521, 528, 541, 561, 590, 591
)

# Flag: 1 = Core course, 0 = Elective
data <- data %>%
  mutate(
    core_flag = as.integer(course_number %in% core_courses)
  )
```

0.7.8 PPMA Student Flag

Identify students likely enrolled in the PPMA program. Eligibility requires taking **2 or more PA courses** across **2 or more distinct terms**.

```
# Flag: PPMA student eligibility
# Rule: took 2 or more PA courses across 2 or more distinct terms
ppma_flags <- data %>%
  filter(course_subject_code == "PA") %>%
  group_by(student_code) %>%
  summarise(
    n_pa_courses = n(),
    n_pa_terms = n_distinct(term_year * 10 + term_season),
    .groups = "drop"
  ) %>%
  mutate(
    ppma_flag = as.integer(n_pa_courses >= 2 & n_pa_terms >= 2)
```

```
)

data <- data %>%
  left_join(ppma_flags %>% select(student_code, ppma_flag),
            by = "student_code") %>%
  mutate(ppma_flag = replace_na(ppma_flag, 0L))
```

0.7.9 PhD Student Flag

Flag students enrolled in PA 599 (PhD dissertation). These students will be excluded from the analysis.

```
# Flag: students enrolled in PA 599 (PhD dissertation → excluded)
data <- data %>%
  group_by(student_code) %>%
  mutate(
    phd_student_flag = as.integer(
      any(course_subject_code == "PA" & course_number == 599)
    )
  ) %>%
  ungroup()
```

0.7.10 Final Inclusion Flag

Determine which students are included in the final analysis dataset.

```
# Flag: final inclusion indicator
# 1 = included in analysis, 0 = excluded
data <- data %>%
  mutate(
    include_flag = as.integer(
      online_only_flag == 0 &
      ppma_flag == 1 &
      phd_student_flag == 0
    )
  )
```

0.8 Exclusion Summary

Summary of how many students were excluded and why.


```
# EXCLUSION SUMMARY (for documentation/reporting)
exclusion_summary <- data %>%
  distinct(
    student_code,
    online_only_flag,
    ppma_flag,
    phd_student_flag,
    include_flag
  ) %>%
  summarise(
    total_students      = n(),
    excluded_online_only = sum(online_only_flag == 1),
    excluded_not_ppma    = sum(
      ppma_flag == 0 &
      online_only_flag == 0 &
      phd_student_flag == 0
    ),
    excluded_phd        = sum(
      phd_student_flag == 1 &
      online_only_flag == 0
    ),
    included_final       = sum(include_flag == 1)
  )
```

0.9 Final Datasets

Export two versions of the dataset:

- **data_full**: All students with all flag variables (for documentation and reporting)
- **data_analysis**: Included students only (`include_flag == 1`), with exclusion flags removed

```
# Full dataset with all students and flags (for documentation/reporting)
data_full <- data
write.csv(data_full, "Data/data_full.csv", row.names = FALSE)

# Analysis dataset: include_flag == 1 only
data_analysis <- data %>%
  filter(include_flag == 1) %>%
  select(-online_only_flag, -ppma_flag, -phd_student_flag, -include_flag)
```

Section 1: Are Students Following the Recommended Course Sequence?

1.1 Exclude Summer Terms

Sequence analysis is based on Fall and Spring semesters only. Summer terms are excluded.

```
data_q1 <- data_analysis %>%  
  filter(term_season != 2)
```

1.2 Extract Student Entry Term

For each student, identify their first observed non-summer enrollment term and season.

```
student_entry <- data_q1 %>%  
  group_by(student_code) %>%  
  summarise(  
    entry_term_code = min(term_code),  
    entry_season    = term_season[term_code == min(term_code)][1],  
    .groups = "drop"  
  )
```

1.3 Define Course Offering Season

Specify which season each foundational course is offered.

```
course_season <- tibble(  
  course_number = c(401, 402, 403),  
  offered_season = c("Fall", "Fall", "Spring")  
)
```

1.4 Filter to Foundational Courses

Keep only PA 401, 402, and 403, retaining only the first time each student took each course.

```
sequence_courses <- c(401, 402, 403)  
  
data_q1 <- data_q1 %>%  
  filter(course_number %in% sequence_courses) %>%  
  arrange(student_code, course_number, term_code) %>%  
  group_by(student_code, course_number) %>%
```

```
slice(1) %>%  
ungroup()
```

1.5 Calculate Opportunity Number

Join entry term and course season information, then calculate the first and second opportunity term codes for each student-course combination.

```
data_q1 <- data_q1 %>%  
  left_join(student_entry, by = "student_code") %>%  
  left_join(course_season, by = "course_number") %>%  
  mutate(  
    first_opportunity_term = case_when(  
      entry_season == 3 & offered_season == "Fall" ~ entry_term_code,  
      entry_season == 1 & offered_season == "Spring" ~ entry_term_code,  
      entry_season == 3 & offered_season == "Spring" ~ entry_term_code + 3,  
      entry_season == 1 & offered_season == "Fall" ~ entry_term_code + 7  
    ),  
    second_opportunity_term = first_opportunity_term + 10,  
    opportunity_number = case_when(  
      term_code == first_opportunity_term ~ 1,  
      term_code == second_opportunity_term ~ 2,  
      TRUE ~ 3  
    )  
  )  
)
```

1.6 Define Adherence

A student is considered adherent if they completed the course within their first two available opportunities.

```
data_q1 <- data_q1 %>%  
  mutate(  
    expected_window = "First two opportunities",  
    adherent_flag = as.integer(opportunity_number <= 2)  
  )
```

1.7 Adherence Rate by Course

```
course_rate <- data_q1 %>%
  group_by(course_number) %>%
  summarise(
    adherence_rate = mean(adherent_flag, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

1.8 Semester Distribution by Course

```
semester_dist <- data_q1 %>%
  mutate(
    semester_category = case_when(
      opportunity_number == 1 ~ "Semester 1",
      opportunity_number == 2 ~ "Semester 2",
      opportunity_number >= 3 ~ "Semester 3+"
    ),
    semester_category = factor(
      semester_category,
      levels = c("Semester 3+", "Semester 2", "Semester 1")
    )
  ) %>%
  group_by(course_number, semester_category) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(course_number) %>%
  mutate(percent = n / sum(n))
```

1.9 Adherence Rate by Curriculum Era

Compare early completion rates before and after the Spring 2025 curriculum change.

```
era_rate <- data_q1 %>%
  group_by(curriculum_era_flag, course_number) %>%
  summarise(
    adherence_rate = mean(adherent_flag, na.rm = TRUE),
    n = n(),
```

```
.groups = "drop"  
)
```

1.10 Prerequisite Analysis: PA 504 and PA 506 Before PA 403

```
prereq_check <- data_analysis %>%  
  filter(term_season != 2) %>%  
  filter(course_number %in% c(403, 504, 506)) %>%  
  arrange(student_code, course_number, term_code) %>%  
  group_by(student_code, course_number) %>%  
  slice(1) %>%  
  ungroup()  
  
prereq_wide <- prereq_check %>%  
  select(student_code, course_number, term_code) %>%  
  pivot_wider(  
    names_from = course_number,  
    values_from = term_code,  
    names_prefix = "term_"  
  )  
  
prereq_analysis <- prereq_wide %>%  
  filter(!is.na(term_403)) %>%  
  mutate(  
    took_504_before_403 = !is.na(term_504) & term_504 < term_403,  
    took_506_before_403 = !is.na(term_506) & term_506 < term_403,  
    took_both_before_403 = took_504_before_403 & took_506_before_403  
  )  
  
prereq_summary <- prereq_analysis %>%  
  summarise(  
    total_took_403 = n(),  
    n_504_before = sum(took_504_before_403),  
    n_506_before = sum(took_506_before_403),  
    n_both_before = sum(took_both_before_403),  
    pct_504_before = mean(took_504_before_403),  
    pct_506_before = mean(took_506_before_403),  
    pct_both_before = mean(took_both_before_403)  
  )
```

Section 2: To What Extent Are Students Taking Multiple Courses Within the Same Semester, and Are There Signs of Course Overload?

2.1 Assign Credit Hours

Assign credit hours to each course. The default is 4 credits unless otherwise specified.

```
data_q2 <- data_analysis %>%  
  mutate(  
    credit_hours = case_when(  
      course_number %in% c(591, 592) ~ 1,  
      course_number %in% c(510, 511, 513, 520, 522, 524, 525, 529,  
                          531, 533, 539, 545, 546, 573, 587,  
                          589, 595) ~ 2,  
      course_number %in% c(447, 577, 426) ~ 3,  
      TRUE ~ 4  
    )  
  )
```

2.2 Remove 1-Credit Courses

1-credit courses are excluded from course load calculations.

```
data_q2_nocredit1 <- data_q2 %>%  
  filter(credit_hours > 1)
```

2.3 Calculate Credit Load per Student per Term

```
credit_load <- data_q2_nocredit1 %>%  
  group_by(student_code, term_year, term_season) %>%  
  summarise(  
    total_credits = sum(credit_hours, na.rm = TRUE),  
    .groups = "drop"  
  )
```

2.4 Calculate Course Count per Student per Term

```
course_load <- data_q2_nocredit1 %>%
  group_by(student_code, term_year, term_season) %>%
  summarise(
    n_courses = n(),
    .groups = "drop"
  )
```

2.5 Remove Summer Terms

Summer terms are excluded for clearer comparison.

```
credit_load_nosummer <- credit_load %>%
  filter(term_season != 2)

course_load_nosummer <- course_load %>%
  filter(term_season != 2)
```

2.6 Create Credit Load Bands

```
credit_load_bands <- credit_load_nosummer %>%
  mutate(
    term_label = case_when(
      term_season == 1 ~ paste("Spring", term_year),
      term_season == 3 ~ paste("Fall", term_year)
    ),
    credit_band = case_when(
      total_credits >= 4 & total_credits <= 5 ~ "4-5 credits",
      total_credits >= 6 & total_credits <= 10 ~ "6-10 credits",
      total_credits >= 11 & total_credits <= 12 ~ "11-12 credits",
      total_credits > 12 ~ ">12 credits",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(credit_band))

term_order_credit <- credit_load_bands %>%
  distinct(term_year, term_season, term_label) %>%
```

```

    arrange(term_year, term_season) %>%
    pull(term_label)

credit_load_bands <- credit_load_bands %>%
  mutate(
    term_label = factor(term_label, levels = term_order_credit),
    credit_band = factor(
      credit_band,
      levels = c(">12 credits", "11-12 credits", "6-10 credits", "4-5 credits")
    )
  )

```

2.7 Summarize Credit Load Bands by Term

```

band_summary <- credit_load_bands %>%
  group_by(term_year, term_season, term_label, credit_band) %>%
  summarise(n_students = n(), .groups = "drop") %>%
  group_by(term_year, term_season, term_label) %>%
  mutate(
    total_students = sum(n_students),
    pct_students = n_students / total_students
  ) %>%
  ungroup()

```

2.8 Create Course Count Bands

```

course_load_bands <- course_load_nosummer %>%
  mutate(
    term_label = case_when(
      term_season == 1 ~ paste("Spring", term_year),
      term_season == 3 ~ paste("Fall", term_year)
    ),
    course_band = case_when(
      n_courses == 1 ~ "1 course",
      n_courses == 2 ~ "2 courses",
      n_courses == 3 ~ "3 courses",
      n_courses >= 4 ~ "4+ courses",
      TRUE ~ NA_character_
    )
  )

```



```

    )
  ) %>%
  filter(!is.na(course_band))

term_order_course <- course_load_bands %>%
  distinct(term_year, term_season, term_label) %>%
  arrange(term_year, term_season) %>%
  pull(term_label)

course_load_bands <- course_load_bands %>%
  mutate(
    term_label = factor(term_label, levels = term_order_course),
    course_band = factor(
      course_band,
      levels = c("4+ courses", "3 courses", "2 courses", "1 course")
    )
  )

```

2.9 Summarize Course Count Bands by Term

```

course_band_summary <- course_load_bands %>%
  group_by(term_year, term_season, term_label, course_band) %>%
  summarise(n_students = n(), .groups = "drop") %>%
  group_by(term_year, term_season, term_label) %>%
  mutate(
    total_students = sum(n_students),
    pct_students = n_students / total_students
  ) %>%
  ungroup()

```

Section 3: How Has Average Time to Degree Completion Changed Over Time?

3.1 Identify PPMA Program Completers

Students are considered program completers if they took a capstone course (PA 590 or PA 592) in their final observed term.

```

# Identify likely PPMA program completers
# Students are retained if they take the capstone course
# (PA 590 or PA 592) in their final observed term.
data_q3 <- data_analysis %>%
  mutate(term_index = term_year * 10 + term_season) %>%
  group_by(student_code) %>%
  summarise(
    start_term          = min(term_index, na.rm = TRUE),
    graduation_term     = max(term_index, na.rm = TRUE),
    last_term_has_capstone = any(
      course_number %in% c(590, 592) &
      term_index == max(term_index, na.rm = TRUE)
    ),
    .groups = "drop"
  ) %>%
  filter(last_term_has_capstone)

```

3.2 Calculate Number of Terms to Degree

Convert term codes into a sequential index (including Spring, Summer, and Fall) to calculate the total number of terms between program entry and graduation.

```

data_q3 <- data_q3 %>%
  mutate(
    start_year  = floor(start_term / 10),
    start_season = start_term %% 10,
    grad_year   = floor(graduation_term / 10),
    grad_season = graduation_term %% 10,

    start_index = start_year * 3 + start_season,
    grad_index  = grad_year * 3 + grad_season,

    n_terms = grad_index - start_index + 1
  )

```

3.3 Assign Graduation Academic Year

Fall, Spring, and Summer terms are grouped into the same academic year (e.g., Fall 2024, Spring 2025, and Summer 2025 are all coded as 2024–2025). Only cohorts from 2018–2019 onward are retained to reduce bias from incomplete early enrollment histories.

```
data_q3 <- data_q3 %>%
  mutate(
    academic_year_start = case_when(
      grad_season == 3 ~ grad_year,
      grad_season %in% c(1, 2) ~ grad_year - 1
    ),
    grad_academic_year = paste0(academic_year_start, "-", academic_year_start + 1)
  ) %>%
  filter(academic_year_start >= 2018)
```

3.4 Summarize Time to Degree by Graduation Cohort

Calculate mean and median number of terms and years to degree for each graduation academic year.

```
terms_to_degree_summary <- data_q3 %>%
  group_by(academic_year_start, grad_academic_year) %>%
  summarise(
    mean_terms = mean(n_terms, na.rm = TRUE),
    median_terms = median(n_terms, na.rm = TRUE),
    mean_years = mean(n_terms / 2, na.rm = TRUE),
    median_years = median(n_terms / 2, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(academic_year_start) %>%
  select(-academic_year_start)
```

3.5 Classify Students by Completion Category

Calculate completion time in years and assign each student to a completion category.

```
data_q3 <- data_q3 %>%
  mutate(
    completion_years = n_terms / 2,

    completion_category = case_when(
      completion_years < 2 ~ "1-<2 years",
      completion_years < 2.5 ~ "2-<2.5 years",
      completion_years <= 3 ~ "2.5-3 years",
    )
  )
```

```

      completion_years <= 4 ~ "3-4 years",
      completion_years <= 5 ~ "4-5 years",
      completion_years > 5 ~ ">5 years"
    )
  )

# Convert completion_category into an ordered factor
data_q3 <- data_q3 %>%
  mutate(
    completion_category = factor(
      completion_category,
      levels = c(
        "1-<2 years",
        "2-<2.5 years",
        "2.5-3 years",
        "3-4 years",
        "4-5 years",
        ">5 years"
      )
    )
  )
)

```

3.6 Distribution by Graduation Academic Year

Count the number of students in each completion category per graduation academic year.

```

distribution_df <- data_q3 %>%
  count(grad_academic_year, completion_category)

```

Section 4: How Have Enrollment Patterns Across Concentrations and Elective Courses Changed Over Time, and What Do These Trends Suggest About Shifts in Student Preferences?

4.1 Prepare Analysis Dataset

```

data_q4 <- data_analysis %>%
  mutate(
    student_code      = as.character(student_code),
    course_subject_code = toupper(as.character(course_subject_code)),
    course_number      = as.numeric(course_number),
    term_year          = as.numeric(term_year),
    term_season        = as.numeric(term_season)
  ) %>%
  filter(!is.na(course_number), course_number >= 400) %>%
  filter(!(course_subject_code == "PA" & course_number == 599))

```

4.2 Standardize Renumbered Courses

```

course_crosswalk_q4 <- tribble(
  ~course_subject_code, ~course_number, ~canonical_course_number,
  "PA", 400, 401,
  "PA", 401, 401,
  "PA", 407, 402,
  "PA", 402, 402,
  "PA", 410, 403,
  "PA", 403, 403,
  "PA", 415, 421,
  "PA", 421, 421,
  "PA", 462, 422,
  "PA", 422, 422,
  "PA", 463, 431,
  "PA", 431, 431,
  "PA", 461, 432,
  "PA", 432, 432,
  "PA", 460, 433,
  "PA", 433, 433,
  "PA", 564, 467,
  "PA", 467, 467,
  "PA", 502, 525,
  "PA", 525, 525,
  "PA", 537, 563,
  "PA", 563, 563,
  "PA", 538, 571,
  "PA", 571, 571,
  "PA", 530, 572,

```

```

"PA", 572, 572,
"PA", 536, 573,
"PA", 573, 573
)

data_q4 <- data_q4 %>%
  left_join(course_crosswalk_q4, by = c("course_subject_code", "course_number")) %>%
  mutate(
    canonical_course_number = coalesce(canonical_course_number, course_number),
    course_id = paste(course_subject_code, canonical_course_number)
  )

```

4.3 Create Academic Year Labels

```

data_q4 <- data_q4 %>%
  mutate(
    academic_year_start = case_when(
      term_season == 3 ~ term_year,
      term_season %in% c(1, 2) ~ term_year - 1,
      TRUE ~ NA_real_
    ),
    academic_year = paste0(academic_year_start, "-", academic_year_start + 1)
  ) %>%
  filter(!is.na(academic_year_start))

academic_year_levels <- data_q4 %>%
  distinct(academic_year, academic_year_start) %>%
  arrange(academic_year_start) %>%
  pull(academic_year)

data_q4 <- data_q4 %>%
  mutate(
    academic_year = factor(academic_year, levels = academic_year_levels)
  )

curriculum_break_position <- which(levels(data_q4$academic_year) == "2024-2025")

```

4.4 Add Course Titles

```
course_lookup <- tribble(
  ~course_subject_code, ~canonical_course_number, ~course_title,
  "PA", 401, "Foundations of Public Service",
  "PA", 402, "Intro to Data Management and Policy",
  "PA", 403, "Economics for Management and Policy",
  "PA", 421, "Public Management",
  "PA", 422, "Project Management",
  "PA", 431, "Civic Tech",
  "PA", 432, "Management of Information Technology",
  "PA", 433, "Data Management",
  "PA", 434, "Intermediate Data Management and Analysis",
  "PA", 435, "GIS for Public Managers",
  "PA", 446, "PA 446",
  "PA", 467, "Civic Engagement",
  "PA", 468, "Topics in Management and Leadership of Public Organizations",
  "PA", 470, "PA 470",
  "PA", 504, "Principles of Financial Management and Budgeting",
  "PA", 505, "Public Management Practices",
  "PA", 506, "Public Policy Development and Process",
  "PA", 515, "PA 515",
  "PA", 521, "Strategic Management: Planning and Measurement",
  "PA", 522, "Ethics and Accountability",
  "PA", 524, "Leadership in Public Organizations",
  "PA", 525, "The Legal Context of Public Administration",
  "PA", 526, "Public Policy Analysis",
  "PA", 528, "PA 528",
  "PA", 529, "Change and Reform in Public Organizations",
  "PA", 531, "Human Resource Management",
  "PA", 532, "Labor Management Relations",
  "PA", 533, "Managing Workplace Diversity",
  "PA", 535, "Conflict Management",
  "PA", 539, "Public Procurement and Contracting",
  "PA", 541, "PA 541",
  "PA", 550, "Advanced Topics in Financial Analysis and Management",
  "PA", 551, "Accounting for Public and Nonprofit Organizations",
  "PA", 552, "Capital Budgeting and Finance",
  "PA", 553, "State and Local Public Finance",
  "PA", 555, "Advanced Topics in Public Budgeting",
  "PA", 561, "Intergovernmental Management",
  "PA", 563, "Local Government Management",
```

```

"PA", 571, "Nonprofit Management",
"PA", 572, "Nonprofit History and Theory of the Nonprofit Sector",
"PA", 573, "Development and Fundraising in Nonprofit Organizations",
"PA", 577, "Survey Questionnaire Design",
"PA", 578, "Surveys, Public Opinion, and Public Policy",
"PA", 579, "Practicum in Survey Methodology",
"PA", 582, "Survey Data Collection Methods",
"PA", 587, "Seminar on Special Topics in Survey Methodology",
"PA", 588, "Applied Survey Sampling and Analysis",
"PA", 590, "Capstone / Portfolio",
"PA", 591, "Managing Your Career",
"PA", 592, "Professional Portfolio",
"POL", 551, "Seminar in Urban Politics",
"POL", 553, "Urban Public Policy",
"UPP", 530, "Economic Development I: Analysis",
"UPP", 462, "UPP 462",
"UPP", 463, "UPP 463",
"UPP", 464, "UPP 464",
"HPA", 407, "Fundamentals of Emergency Management",
"EOHS", 572, "Environmental Risk Assessment and Management",
"IDS", 509, "IDS 509",
"IDS", 524, "Strategic Emergency Management",
"IDS", 541, "Disaster Response and Recovery Operations",
"IDS", 576, "IDS 576",
"BSTT", 426, "BSTT 426",
"BSTT", 537, "BSTT 537",
"EPSY", 550, "EPSY 550",
"EPSY", 551, "EPSY 551",
"EPSY", 584, "EPSY 584",
"EPSY", 585, "EPSY 585",
"EPSY", 587, "EPSY 587",
"PPOL", 405, "PPOL 405"
)

data_q4 <- data_q4 %>%
  left_join(course_lookup, by = c("course_subject_code", "canonical_course_number")) %>%
  mutate(
    course_title = coalesce(
      course_title,
      paste(course_subject_code, canonical_course_number)
    ),
    course_label = paste0(course_id, ": ", course_title)
  )

```



```
)
```

4.5 Separate Core and Elective Courses

```
core_courses_q4 <- c(
  "PA 401", "PA 402", "PA 403",
  "PA 504", "PA 505", "PA 506",
  "PA 521", "PA 561", "PA 590", "PA 591", "PA 592"
)

data_q4 <- data_q4 %>%
  mutate(
    course_group = if_else(
      course_id %in% core_courses_q4,
      "Core / Required",
      "Elective / Concentration"
    )
  )

electives <- data_q4 %>%
  filter(course_group == "Elective / Concentration")
```

4.6 Assign Topic Areas

```
topic_map <- tribble(
  ~course_subject_code, ~canonical_course_number, ~topic,
  "PA", 412, "Methods / Data / Analytics",
  "PA", 431, "Methods / Data / Analytics",
  "PA", 432, "Methods / Data / Analytics",
  "PA", 433, "Methods / Data / Analytics",
  "PA", 434, "Methods / Data / Analytics",
  "PA", 435, "Methods / Data / Analytics",
  "PA", 446, "Methods / Data / Analytics",
  "PA", 447, "Methods / Data / Analytics",
  "PA", 470, "Methods / Data / Analytics",
  "PA", 482, "Methods / Data / Analytics",
  "PA", 520, "Methods / Data / Analytics",
  "PA", 528, "Methods / Data / Analytics",
)
```

"PA", 540, "Methods / Data / Analytics",
 "PA", 541, "Methods / Data / Analytics",
 "PA", 542, "Methods / Data / Analytics",
 "PA", 543, "Methods / Data / Analytics",
 "PA", 544, "Methods / Data / Analytics",
 "PA", 545, "Methods / Data / Analytics",
 "PA", 546, "Methods / Data / Analytics",
 "PA", 577, "Methods / Data / Analytics",
 "PA", 578, "Methods / Data / Analytics",
 "PA", 579, "Methods / Data / Analytics",
 "PA", 582, "Methods / Data / Analytics",
 "PA", 587, "Methods / Data / Analytics",
 "PA", 588, "Methods / Data / Analytics",
 "PA", 589, "Methods / Data / Analytics",
 "PA", 593, "Methods / Data / Analytics",
 "PA", 596, "Methods / Data / Analytics",
 "UPP", 462, "Methods / Data / Analytics",
 "UPP", 463, "Methods / Data / Analytics",
 "UPP", 464, "Methods / Data / Analytics",
 "IDS", 509, "Methods / Data / Analytics",
 "IDS", 576, "Methods / Data / Analytics",
 "BSTT", 426, "Methods / Data / Analytics",
 "BSTT", 537, "Methods / Data / Analytics",
 "EPSY", 550, "Methods / Data / Analytics",
 "EPSY", 551, "Methods / Data / Analytics",
 "EPSY", 584, "Methods / Data / Analytics",
 "EPSY", 585, "Methods / Data / Analytics",
 "EPSY", 587, "Methods / Data / Analytics",
 "PPOL", 405, "Methods / Data / Analytics",
 "PA", 504, "Finance / Budgeting",
 "PA", 550, "Finance / Budgeting",
 "PA", 551, "Finance / Budgeting",
 "PA", 552, "Finance / Budgeting",
 "PA", 553, "Finance / Budgeting",
 "PA", 554, "Finance / Budgeting",
 "PA", 555, "Finance / Budgeting",
 "PA", 531, "Nonprofit",
 "PA", 551, "Nonprofit",
 "PA", 571, "Nonprofit",
 "PA", 572, "Nonprofit",
 "PA", 573, "Nonprofit",
 "PA", 452, "Governance / Policy / Urban",

"PA", 453, "Governance / Policy / Urban",
 "PA", 466, "Governance / Policy / Urban",
 "PA", 467, "Governance / Policy / Urban",
 "PA", 515, "Governance / Policy / Urban",
 "PA", 526, "Governance / Policy / Urban",
 "PA", 528, "Governance / Policy / Urban",
 "PA", 561, "Governance / Policy / Urban",
 "PA", 562, "Governance / Policy / Urban",
 "PA", 563, "Governance / Policy / Urban",
 "PA", 565, "Governance / Policy / Urban",
 "PA", 567, "Governance / Policy / Urban",
 "POLS", 551, "Governance / Policy / Urban",
 "POLS", 553, "Governance / Policy / Urban",
 "UPP", 530, "Governance / Policy / Urban",
 "PPOL", 405, "Governance / Policy / Urban",
 "PA", 421, "Management / Leadership / Organizations",
 "PA", 422, "Management / Leadership / Organizations",
 "PA", 468, "Management / Leadership / Organizations",
 "PA", 490, "Management / Leadership / Organizations",
 "PA", 505, "Management / Leadership / Organizations",
 "PA", 510, "Management / Leadership / Organizations",
 "PA", 511, "Management / Leadership / Organizations",
 "PA", 513, "Management / Leadership / Organizations",
 "PA", 521, "Management / Leadership / Organizations",
 "PA", 522, "Management / Leadership / Organizations",
 "PA", 524, "Management / Leadership / Organizations",
 "PA", 525, "Management / Leadership / Organizations",
 "PA", 527, "Management / Leadership / Organizations",
 "PA", 529, "Management / Leadership / Organizations",
 "PA", 531, "Management / Leadership / Organizations",
 "PA", 532, "Management / Leadership / Organizations",
 "PA", 533, "Management / Leadership / Organizations",
 "PA", 534, "Management / Leadership / Organizations",
 "PA", 535, "Management / Leadership / Organizations",
 "PA", 539, "Management / Leadership / Organizations",
 "PA", 594, "Management / Leadership / Organizations",
 "HPA", 407, "Emergency / Resilience",
 "EOHS", 572, "Emergency / Resilience",
 "IDS", 524, "Emergency / Resilience",
 "IDS", 541, "Emergency / Resilience"

)

```
topic_data <- electives %>%
  inner_join(topic_map, by = c("course_subject_code", "canonical_course_number"))
```

4.7 Calculate Percent of Students Enrolled in Each Topic by Academic Year

```
total_students_ay <- data_q4 %>%
  distinct(student_code, academic_year, academic_year_start) %>%
  count(academic_year, academic_year_start, name = "total_students") %>%
  arrange(academic_year_start)

topic_students <- topic_data %>%
  distinct(student_code, academic_year, academic_year_start, topic) %>%
  count(academic_year, academic_year_start, topic, name = "students_in_topic") %>%
  arrange(academic_year_start, topic)

topic_percent <- topic_students %>%
  complete(
    academic_year,
    topic,
    fill = list(students_in_topic = 0)
  ) %>%
  left_join(
    total_students_ay %>% select(academic_year, academic_year_start, total_students),
    by = "academic_year"
  ) %>%
  mutate(
    percent_students = if_else(
      !is.na(total_students) & total_students > 0,
      students_in_topic / total_students,
      0
    )
  )
```

4.8 Count Courses Offered by Topic and Academic Year

```
courses_offered <- topic_data %>%
  distinct(academic_year, topic, course_id) %>%
  count(academic_year, topic, name = "courses_offered") %>%
```

```
complete(
  academic_year,
  topic,
  fill = list(courses_offered = 0)
)
```

Section 5: To What Extent Do Students Deviate From the Recommended Course Plan Across the Three Programs (MPA, MPP, and MSCA)?

5.1 Prepare Analysis Dataset

Apply additional filters required for Q5.

```
data_q5 <- data_analysis %>%
  mutate(
    student_code      = as.character(student_code),
    course_subject_code = toupper(as.character(course_subject_code)),
    course_number      = as.numeric(course_number)
  ) %>%
  filter(!is.na(student_code), !is.na(course_number), course_number >= 400) %>%
  filter(!(course_subject_code == "PA" & course_number == 599))
```

5.2 Standardize Renumbered Courses

Map old course numbers to their canonical course numbers and create a unique course ID.

```
course_crosswalk_q5 <- tribble(
  ~course_number, ~canonical_course_number,
  400, 401,
  401, 401,
  407, 402,
  402, 402,
  410, 403,
  403, 403,
  415, 421,
  421, 421,
  462, 422,
  422, 422,
```

```

463, 431,
431, 431,
461, 432,
432, 432,
460, 433,
433, 433,
564, 467,
467, 467,
502, 525,
525, 525,
537, 563,
563, 563,
538, 571,
571, 571,
530, 572,
572, 572,
536, 573,
573, 573
)

data_q5 <- data_q5 %>%
  left_join(course_crosswalk_q5, by = "course_number") %>%
  mutate(
    canonical_course_number = if_else(
      is.na(canonical_course_number),
      course_number,
      canonical_course_number
    ),
    course_id = paste(course_subject_code, canonical_course_number)
  )

```

5.3 Identify Valid PPMA Students

Retain students who took PA 401 or at least 3 distinct PA courses.

```

ppma_check <- data_q5 %>%
  filter(course_subject_code == "PA") %>%
  group_by(student_code) %>%
  summarise(
    pa_courses = n_distinct(course_id),
    took_foundation = any(course_id == "PA 401"),
  )

```

```

    .groups = "drop"
  )

valid_students <- ppma_check %>%
  filter(took_foundation | pa_courses >= 3) %>%
  pull(student_code)

data_q5 <- data_q5 %>%
  filter(student_code %in% valid_students)

```

5.4 Assign Students to Programs

Infer each student's program (MPA, MPP, or MSCA) based on the program-signature courses they took. Each student is assigned to the program with the highest number of matching signature courses.

```

program_signatures <- tribble(
  ~program, ~course_id,
  "MPA", "PA 504",
  "MPA", "PA 505",
  "MPA", "PA 561",
  "MPP", "PA 515",
  "MPP", "PA 528",
  "MPP", "PA 541",
  "MSCA", "PA 431",
  "MSCA", "PA 434",
  "MSCA", "PA 446"
)

student_program <- data_q5 %>%
  distinct(student_code, course_id) %>%
  inner_join(program_signatures, by = "course_id") %>%
  count(student_code, program) %>%
  group_by(student_code) %>%
  slice_max(n, n = 1, with_ties = FALSE) %>%
  ungroup()

# Keep only students with an identifiable program
data_q5 <- data_q5 %>%
  left_join(student_program, by = "student_code") %>%
  filter(!is.na(program))

```

5.5 Define Required and Concentration Courses

Specify the required courses for each program and the list of recognized concentration courses.

```
mpa_required <- c(
  "PA 401", "PA 402", "PA 403",
  "PA 504", "PA 505", "PA 506",
  "PA 521", "PA 561", "PA 590", "PA 591"
)

mpp_required <- c(
  "PA 401", "PA 402", "PA 403",
  "PA 434", "PA 506", "PA 521", "PA 590", "PA 591",
  "PA 515", "PA 528", "PA 541", "PA 526"
)

msca_required <- c(
  "PA 401", "PA 402", "PA 403",
  "PA 506", "PA 521", "PA 590", "PA 591",
  "PA 431", "PA 434", "PA 446", "PA 470", "PA 541"
)

concentration_courses <- c(
  "PA 422", "PA 431", "PA 434", "PA 467", "PA 550", "PA 551",
  "PA 552", "PA 553", "PA 555", "PA 563", "PA 571", "PA 582",
  "UPP 530", "POLS 551", "POLS 553",
  "PA 468", "PA 526", "PA 521", "PA 535", "PA 539"
)
```

5.6 Flag On-Plan and Off-Plan Courses

A course is considered on-plan if it is part of the student's required courses or a recognized concentration course. All other courses are flagged as off-plan.

```
data_q5 <- data_q5 %>%
  mutate(
    on_plan = case_when(
      program == "MPA" & course_id %in% mpa_required ~ 1,
      program == "MPP" & course_id %in% mpp_required ~ 1,
      program == "MSCA" & course_id %in% msca_required ~ 1,
      course_id %in% concentration_courses ~ 1,
    )
  )
```



```

    TRUE ~ 0
  ),
  off_plan = if_else(on_plan == 1, 0, 1)
)

```

5.7 Summarize Off-Plan Courses per Student

Count the number of off-plan and total courses taken by each student within their assigned program.

```

student_offplan <- data_q5 %>%
  distinct(student_code, program, course_id, off_plan) %>%
  group_by(student_code, program) %>%
  summarise(
    off_plan_courses = sum(off_plan),
    total_courses    = n_distinct(course_id),
    .groups = "drop"
  )

```

5.8 Summary Table by Program

```

summary_table_q5 <- student_offplan %>%
  group_by(program) %>%
  summarise(
    students      = n(),
    mean_off_plan = mean(off_plan_courses),
    median_off_plan = median(off_plan_courses),
    max_off_plan  = max(off_plan_courses),
    .groups = "drop"
  )

```

Section 6: How Has Student Participation in Nonprofit Coursework Changed Over Time, and What Explains These Trends?

6.1 Create Academic Year Variable

Group terms into academic years. Fall starts a new academic year; Spring and Summer belong to the prior Fall.

```
data_q6 <- data_analysis %>%
  mutate(
    academic_year = case_when(
      term_season == 3 ~ paste0(term_year, "-", term_year + 1),
      term_season %in% c(1, 2) ~ paste0(term_year - 1, "-", term_year)
    )
  )

# Fix proper chronological ordering
year_levels <- data_q6 %>%
  distinct(academic_year) %>%
  arrange(academic_year) %>%
  pull(academic_year)

data_q6 <- data_q6 %>%
  mutate(academic_year = factor(academic_year, levels = year_levels))
```

6.2 Share of Nonprofit Enrollments by Academic Year

Calculate the percentage of total course enrollments that were in nonprofit courses for each academic year.

```
nonprofit_by_year <- data_q6 %>%
  group_by(academic_year) %>%
  summarise(
    pct_nonprofit = mean(nonprofit_flag, na.rm = TRUE) * 100,
    .groups = "drop"
  )
```

6.3 Number of Nonprofit Course Offerings by Academic Year

Count the number of distinct nonprofit course sections (CRNs) offered in each academic year.

```

nonprofit_courses_offered <- data_q6 %>%
  filter(nonprofit_flag == 1) %>%
  group_by(academic_year) %>%
  summarise(
    total_courses = n_distinct(crn),
    .groups = "drop"
  )

```

6.4 Total and Nonprofit Enrollment Counts by Academic Year

```

enrollment_counts <- data_q6 %>%
  group_by(academic_year) %>%
  summarise(
    total_enrollments = n(),
    nonprofit_enrollments = sum(nonprofit_flag, na.rm = TRUE),
    .groups = "drop"
  )

```

6.5 Summary Table

```

nonprofit_table <- nonprofit_by_year %>%
  left_join(nonprofit_courses_offered, by = "academic_year") %>%
  left_join(enrollment_counts, by = "academic_year") %>%
  mutate(
    total_courses = ifelse(is.na(total_courses), 0, total_courses),
    pct_nonprofit = round(pct_nonprofit, 1),
    academic_year = as.character(academic_year)
  ) %>%
  select(
    academic_year,
    total_courses,
    nonprofit_enrollments,
    total_enrollments,
    pct_nonprofit
  ) %>%
  rename(
    "Academic Year" = academic_year,
    "Number of Nonprofit Course Offerings" = total_courses,
  )

```

```

    "Number of Nonprofit Enrollments" = nonprofit_enrollments,
    "Total Enrollments"              = total_enrollments,
    "Percent of Nonprofit Enrollments" = pct_nonprofit
  )

```

Section 7: To What Extent Do Students Retake Courses, and Which Courses Are Most Frequently Repeated?

7.1 Create Sequential Term Order

Build a lookup table that assigns a sequential order to each term for calculating gaps between retakes.

```

data_q7 <- data_analysis %>%
  mutate(term_index = term_year * 10 + term_season)

term_lookup <- data_q7 %>%
  distinct(term_year, term_season, term_index) %>%
  arrange(term_year, term_season) %>%
  mutate(term_seq = row_number())

data_q7 <- data_q7 %>%
  left_join(term_lookup, by = c("term_year", "term_season", "term_index"))

```

7.2 Identify Course Retakes

For each student-course combination, calculate the first and last term taken, the number of distinct terms, and the gap between first attempt and retake. Only students who took the same course in more than one term are retained.

```

retake_detail <- data_q7 %>%
  group_by(student_code, course_number) %>%
  summarise(
    first_term_seq = min(term_seq, na.rm = TRUE),
    last_term_seq  = max(term_seq, na.rm = TRUE),
    first_year     = term_year[which.min(term_seq)],
    first_season   = term_season[which.min(term_seq)],
    last_year      = term_year[which.max(term_seq)],

```

```

    last_season      = term_season[which.max(term_seq)],
    n_terms_taken    = n_distinct(term_seq),
    retake_flag      = n_terms_taken > 1,
    .groups = "drop"
  ) %>%
  mutate(
    gap_in_semesters = last_term_seq - first_term_seq
  ) %>%
  filter(retake_flag)

```

7.3 Summarize Retakes at the Student Level

For each student, calculate the total number of retaken courses, the list of retaken course numbers, and the average and maximum gap between attempts.

```

student_retake <- retake_detail %>%
  group_by(student_code) %>%
  summarise(
    any_retake      = TRUE,
    n_retake_courses = n(),
    retaken_courses = paste(sort(unique(course_number)), collapse = ", "),
    avg_gap_semesters = mean(gap_in_semesters, na.rm = TRUE),
    max_gap_semesters = max(gap_in_semesters, na.rm = TRUE),
    .groups = "drop"
  )

```

7.4 Merge Retake Information with Student Dataset

```

data_q7_students <- data_q3 %>%
  left_join(student_retake, by = "student_code") %>%
  mutate(
    any_retake = ifelse(is.na(any_retake), FALSE, any_retake)
  )

```

7.5 Retake Share Summary

Calculate the share of students who retook at least one course.

```

retake_share <- data_q7_students %>%
  count(any_retake) %>%
  mutate(
    label      = ifelse(any_retake, "Retake", "No Retake"),
    percent    = n / sum(n),
    label_text = paste0(label, "\n", round(percent * 100, 1), "%")
  )

```

7.6 Most Frequently Retaken Courses

Count how many times each course (400-level or higher) was retaken across all students.

```

retake_by_course <- retake_detail %>%
  filter(course_number >= 400) %>%
  count(course_number, name = "n_retakes") %>%
  arrange(desc(n_retakes))

```